

William Anderson

Senior Software Engineer

Seminole, FL | williamandresondev@outlook.com | +1 (407) 801-1287 |
<https://www.linkedin.com/in/william-andreson-39b88b3aa>

Professional Summary

Senior Software Engineer with 10+ years of experience delivering **Python** and **React** solutions across SaaS, operations, and cloud-based product environments, combining **Python 2.7–3.12**, **React 16–19**, **TypeScript**, **FastAPI**, **Django**, **Flask**, and modern API design to build secure, scalable, and maintainable platforms for the **software, digital product, and business operations industry**. Strong background in debugging production issues, modernizing legacy systems, improving release reliability, and shipping user-facing features that increased operational efficiency, reduced failure rates by up to **34%**, and improved response times by up to **41%** through practical architecture, clean service boundaries, and flexible full-stack delivery.

Technical Skills

- **Frontend:** React 16–19, Next.js 12–15, TypeScript, JavaScript ES6+, HTML5, CSS3, Tailwind CSS, Redux Toolkit, Context API, React Query, Vite, Webpack
- **Backend & APIs:** Python 2.7–3.12, FastAPI, Django, Flask, REST API design, GraphQL, OpenAPI, JWT authentication, RBAC, background jobs, microservices, Celery
- **Data & Messaging:** PostgreSQL, MySQL, SQL Server, Redis, MongoDB, Elasticsearch, RabbitMQ, Kafka
- **Cloud & DevOps:** AWS, EC2, ECS, EKS, Lambda, API Gateway, S3, RDS, CloudFront, Docker, Kubernetes, Terraform, GitHub Actions, Jenkins, CI/CD
- **Observability, Quality & Security:** Pytest, Jest, React Testing Library, Cypress, Playwright, OpenTelemetry, Prometheus, Grafana, Datadog, structured logging, audit logging, rate limiting, OAuth2, OWASP security practices

Professional Experience

Viaro Networks Inc.

Senior Software Engineer | January 2021 – December 2025

- Led delivery of cloud-hosted product features using **Python 3.10–3.12**, **FastAPI**, **React 18–19**, and **TypeScript**, building full-stack workflows that supported high-volume internal and client-facing operations with stronger reliability and cleaner service boundaries.
- Designed and implemented modular backend services for account workflows, reporting pipelines, permissions, and event-driven processing, helping the platform handle growing demand while reducing repeat support escalations by **29%** across several operational teams.
- Modernized a legacy frontend experience by moving critical screens into **React** with reusable component patterns, improving maintainability, reducing UI regression frequency, and cutting average feature delivery time by **24%** for shared product modules.
- Resolved a recurring production issue where long-running export requests caused request timeouts and duplicate user actions, then moved heavy processing into asynchronous workers with status polling so completion reliability improved by **34%**.

- Introduced **React Query** caching and better query invalidation patterns for data-heavy dashboards, which reduced unnecessary API traffic and improved page responsiveness by **31%** for frequently accessed operational views.
- Refactored older Python services into clearer domain-based modules with stronger validation, consistent error handling, and shared response contracts, making debugging easier and shortening mean time to resolution during incidents.
- Implemented role-aware API authorization with **JWT** and **RBAC**, aligning backend enforcement with frontend visibility rules so privileged actions became more secure and audit findings around permission drift were reduced.
- Migrated selected services from older **Flask** patterns into **FastAPI** to improve schema validation, API documentation quality, and maintainability, while preserving compatibility through staged rollout and backward-safe endpoint support.
- Fixed a memory growth issue caused by unbounded in-process caching and stale event listeners in background workers, then replaced fragile logic with bounded cache rules and safer lifecycle cleanup to stabilize container performance.
- Built observability improvements using structured logging, metrics, and alert-friendly error classification, which gave engineers clearer production signals and helped reduce investigation time by **27%** during peak-load incidents.
- Partnered with product and design teams to implement new self-service workflow features in **React** that reduced manual back-office handling, improved completion quality, and raised internal adoption of the updated tools.
- Strengthened CI/CD quality gates with automated backend tests, frontend checks, and contract validation for critical APIs, reducing avoidable deployment regressions and making releases more predictable across shared environments.

VividCloud

Software Engineer | January 2016 – December 2020

- Built business-critical web applications using **Python 3.x**, **Django**, **Flask**, **React 16–18**, and relational databases, delivering dependable internal systems and customer-facing modules for teams that relied on stable day-to-day workflows.
- Implemented API endpoints, validation rules, reporting logic, and operational dashboards that improved process consistency and gave users more responsive tools for handling account, transaction, and service-related tasks.
- Contributed to frontend modernization by introducing **React** into older server-rendered areas through incremental migration, creating a lower-risk path that improved maintainability without requiring disruptive platform-wide rewrites.
- Corrected a high-impact issue in reporting screens where repeated joins and weak filtering logic produced inaccurate totals under edge-case conditions, then tightened service rules and query structure to restore output trust.
- Reworked slow list and search pages by introducing pagination, selective field loading, and practical indexing strategies, which improved response times by **38%** on several high-traffic administrative views.
- Added asynchronous job handling for import, export, and reconciliation routines so long-running tasks no longer blocked web requests, leading to fewer abandoned actions and more predictable task completion behavior.
- Implemented reusable error-handling and response-format utilities across Python services, helping the team reduce inconsistent API behavior and making frontend integration work far smoother for new feature development.
- Helped migrate older frontend logic into component-based **React** modules with clearer state handling, reducing duplicated UI code and improving defect isolation when changes were introduced during active release cycles.

- Improved release safety by expanding automated testing around critical service logic and UI flows, which reduced production defects after deployment and increased team confidence in incremental changes.
- Integrated audit logging for sensitive record changes and administrative actions, making compliance review easier and giving support teams stronger visibility when investigating customer-facing issues.
- Worked closely with stakeholders to implement new operational features based on real support pain points, contributing to measurable workflow improvements and reducing manual correction effort by **22%**.

Minimalist Moon

Software Developer | *January 2013 – December 2015*

- Developed internal software and client-facing business tools using **Python 2.7–3.x**, **Flask**, early **React 15–16** adoption for selective modules, and relational database workflows suited to the delivery patterns widely used during that period.
- Built core CRUD features, approval flows, reporting logic, and role-based business rules that gave operations teams more dependable systems and reduced spreadsheet-heavy manual handling in everyday processes.
- Introduced targeted frontend enhancements with **React** on high-value screens while maintaining compatibility with server-rendered templates, helping the team modernize gradually without creating unnecessary delivery risk.
- Fixed a recurring defect where form resubmissions during network instability created duplicate records, then added safer backend validation and idempotent handling patterns to protect business-critical operations.
- Created CSV import and export utilities with field-level validation and user-friendly feedback, improving first-pass data quality and reducing data-cleanup effort by **26%** for support and operations users.
- Refined older SQL queries and page-loading patterns by trimming payload size, improving joins, and using more practical database indexes, which improved screen performance by **33%** on commonly used internal pages.
- Added better exception logging and environment-based configuration practices so issues could be reproduced more reliably across development and staging, making troubleshooting much less guess-driven for the engineering team.
- Helped implement new dashboard and activity-tracking features that gave managers better operational visibility, improving turnaround planning and reducing follow-up gaps in workflow-heavy processes.
- Supported a gradual backend cleanup effort that separated business logic from route handlers, improving readability and making future migrations toward more maintainable service patterns easier for the team.
- Delivered bug fixes and incremental UX improvements across both backend and frontend layers, showing flexibility in full-stack delivery while helping stabilize a platform that was still evolving rapidly.

Microsoft

Software Developer Intern | *July 2012 – December 2012*

- Supported application development tasks across backend services, UI enhancements, debugging, and internal tooling work, gaining practical exposure to production-minded engineering standards and collaborative delivery practices.
- Contributed to feature updates by implementing small but meaningful improvements in validation, interface behavior, and data handling, helping senior engineers move larger initiatives forward with fewer low-level blockers.
- Investigated defects reported during testing and internal usage, tracing failures through logs, reproducible steps, and code review so fixes could be delivered with stronger confidence and less rework.

- Assisted with code cleanup, test support, and documentation updates that made onboarding easier for teammates and improved clarity around service behavior, expected inputs, and known operational constraints.
- Worked within existing engineering practices for source control, reviews, and release preparation, building a strong foundation in maintainable coding habits and quality-focused team collaboration.
- Implemented small internal utilities and workflow improvements that reduced repetitive manual steps for developers and testers, demonstrating adaptable problem-solving and reliable execution in a professional environment.
- Helped validate fixes across development scenarios and edge cases so release candidates behaved more predictably before promotion to shared environments used by larger internal teams.
- Learned to balance speed and correctness while collaborating across engineering stakeholders, contributing to a stronger understanding of how production software is built, reviewed, and improved.
- Supported incremental improvements to older modules by making low-risk changes with clear reviewability, which helped reduce technical friction while respecting the constraints of an actively used codebase.

Education

Florida State University

Bachelor's Degree in Computer Science | 2008 – 2012